



Curso Javascript básico

Aula 02 - Operadores



O que são operadores?

Operadores em Javascript são símbolos especiais que envolvem um ou mais operandos com a finalidade de produzir um determinado resultado.

O JavaScript possui tanto operadores binários quanto unários e um operador ternário, o operador condicional.

O **operador binário** exige dois operandos, um antes do operador e outro depois: Por exemplo, $3+4$ ou $x*y$;

O **operador unário** exige um único operando, seja antes ou depois do operador: Por exemplo, $x++$ ou $++x$;

O **operador condicional (ternário)** é o único operador JavaScript que utiliza três operadores. operador pode ter um de dois valores em uma condição: Por exemplo, `var status = (idade >= 18) ? "adulto" : "menor de idade"`.



Tipos de operadores

Operador de atribuição

Um operador de atribuição atribui um valor ao operando à sua esquerda baseado no valor do operando à direita. O operador de atribuição básico é o igual (=), que atribui o valor do operando à direita ao operando à esquerda.

Isto é, $x = y$ atribui o valor de y a x , ou, $x = 5$ atribui o valor de 5 a x .

Existem também os operadores de atribuição composta que serão explicados com exemplos.

Atribuição composta de soma (+=)

```
x += 10;
```

O código acima irá verificar o valor de x , somá-lo a 10 e atribuí-lo novamente a x . Por exemplo, se o valor de x for 5, será feita a soma de 5 com 10 e o valor final, 15, será atribuído novamente a variável x .



Tipos de operadores

Operador de atribuição

Atribuição composta de subtração (-=)

```
x -= 3;
```

Como na adição composta, a subtração irá considerar o valor da variável x, subtrair dela o segundo operando, 3, e armazená-lo de volta na variável x.

Atribuição composta de multiplicação (*=)

```
x *= 8;
```

Você já deve ter entendido como é o funcionamento da composição do operador de atribuição com operadores aritméticos. No código acima, portanto, o valor de x será multiplicado por 8 e atribuído novamente a x.



Tipos de operadores

Operador de atribuição

Atribuição composta de divisão (/=) e resto (%=)

```
x /= 2;  x %= 3;
```

Por fim, a atribuição composta de divisão e resto armazenará na variável x o resultado da divisão de x por 2, e o resto da divisão de x por 3, respectivamente.

Esses operadores compostos são uma contração de uma operação aritmética e outra de atribuição. Por exemplo, a instrução:

```
p += 7;
```

é equivalente a:

```
p = p + 7;
```



Curso Javascript básico – Aula 02

Nome	Operador encurtado	Significado
Atribuição	<code>x = y</code>	<code>x = y</code>
Atribuição de adição	<code>x += y</code>	<code>x = x + y</code>
Atribuição de subtração	<code>x -= y</code>	<code>x = x - y</code>
Atribuição de multiplicação	<code>x *= y</code>	<code>x = x * y</code>
Atribuição de divisão	<code>x /= y</code>	<code>x = x / y</code>
Atribuição de resto	<code>x %= y</code>	<code>x = x % y</code>
Atribuição exponencial	<code>x **= y</code>	<code>x = x ** y</code>
Atribuição bit-a-bit por deslocamento á esquerda	<code>x <<= y</code>	<code>x = x << y</code>
Atribuição bit-a-bit por deslocamento á direita	<code>x >>= y</code>	<code>x = x >> y</code>
Atribuição de bit-a-bit deslocamento á direita não assinado	<code>x >>>= y</code>	<code>x = x >>> y</code>
Atribuição AND bit-a-bit	<code>x &= y</code>	<code>x = x & y</code>
Atribuição XOR bit-a-bit	<code>x ^= y</code>	<code>x = x ^ y</code>
Atribuição OR bit-a-bit	<code>x = y</code>	<code>x = x y</code>



Tipos de operadores

Operador de comparação

Os operadores de comparação envolvem dois operandos e retornam um resultado lógico. O operador de igualdade (`==`), por exemplo, retorna verdadeiro (`true`), caso os operandos tenham o valor igual, e retornará falso (`false`), caso sejam diferentes.

Vamos ver em detalhes quais são os operadores de comparação que podemos usar quando programamos em Javascript:



Tipos de operadores

Operador de comparação

Igual (==)

O operador de igualdade compara dois operandos e retorna verdadeiro, se eles forem iguais, e falso, se forem diferentes. Veja os exemplos abaixo:

```
5 == 5; // retornará verdadeiro  
5 == '5' // também retornará verdadeiro  
5 == 4 // retornará falso  
5 == 'c' // também retornará falso
```

Observe que para o Javascript não importa se os operandos recebidos pelo operador de igualdade são do mesmo tipo. No exemplo acima, o Javascript faz a conversão (casting) de um dos operandos e depois faz a comparação.



Tipos de operadores

Operador de comparação

Diferente (!=)

Esse operador compara os dois operandos e retorna verdadeiro, se eles forem diferentes, e falso, se forem iguais. A lógica é a inversa do operador de igualdade.

'string1' != 'string2' // retorna verdadeiro

O exemplo acima também serve para mostrar que além de caracteres, os operadores de comparação também podem ser aplicados a strings.



Tipos de operadores

Operador de comparação

Igualdade estrita (===)

A igualdade estrita, além de considerar o valor dos operandos, leva em conta também seu tipo:

```
100 === 100; // retorna true
```

```
100 === '100' // retorna false
```

```
100 === 3 // também retorna false
```



Tipos de operadores

Operador de comparação

Diferença estrita (!==)

Tem funcionamento análogo à diferença (!=), no entanto, considera se o tipo dos operandos é o mesmo, como o operador de igualdade estrita.

```
100 !== 100; // retorna false  
100 !== '100' // retorna true  
100 !== 3 // também retorna true
```



Tipos de operadores

Operador de comparação

Maior (>)

É usado para indicar se o operador da esquerda é maior que o operador da direita:

5 > 4 // retornará verdadeiro

Menor (<)

Retorna verdadeiro se o operador da esquerda for menor que o operador da direita:

6 < 10 // retornará verdadeiro



Tipos de operadores

Operador de comparação

Maior ou igual ($\geq$)

Seu funcionamento é semelhante ao operador 'maior', mas retorna verdadeiro caso o operador da esquerda seja maior ou igual ao operador da direita. Veja:

6 $\geq$ 4 // retorna verdadeiro

6 $\geq$ 6 // também retorna verdadeiro

4 $\geq$ 6 // retorna falso



Tipos de operadores

Operador de comparação

Menor ou igual (<=)

Partindo da declaração das variáveis acima, vamos analisar o seguinte trecho de código:

```
6 <= 4 // retorna falso
```

Essa instrução retornará falso, já que 6 é maior que 4. Por outro lado, se invertermos os operandos, a instrução abaixo retornará true.

```
4 <= 6 // retorna verdadeiro
```



Tipos de operadores

Operador de comparação

Menor ou igual (<=)

Partindo da declaração das variáveis acima, vamos analisar o seguinte trecho de código:

```
6 <= 4 // retorna falso
```

Essa instrução retornará falso, já que 6 é maior que 4. Por outro lado, se invertermos os operandos, a instrução abaixo retornará true.

```
4 <= 6 // retorna verdadeiro
```



Operadores de comparação

Operador	Descrição	Exemplos que retornam verdadeiro
Igual (==)	Retorna verdadeiro caso os operandos sejam iguais.	<code>3 == var1 "3" == var1 3 == '3'</code>
Não igual (!=)	Retorna verdadeiro caso os operandos não sejam iguais.	<code>var1 != 4 var2 != "3"</code>
Estritamente igual (===)	Retorna verdadeiro caso os operandos sejam iguais e do mesmo tipo.	<code>3 === var1</code>
Estritamente não igual (!==)	Retorna verdadeiro caso os operandos não sejam iguais e/ou não sejam do mesmo tipo.	<code>var1 !== "3" 3 !== '3'</code>
Maior que (>)	Retorna verdadeiro caso o operando da esquerda seja maior que o da direita.	<code>var2 > var1 "12" > 2</code>
Maior que ou igual (>=)	Retorna verdadeiro caso o operando da esquerda seja maior ou igual ao da direita.	<code>var2 >= var1 var1 >= 3</code>
Menor que (<)	Retorna verdadeiro caso o operando da esquerda seja menor que o da direita.	<code>var1 < var2 "12" < "2"</code>
Menor que ou igual (<=)	Retorna verdadeiro caso o operando da esquerda seja menor ou igual ao da direita.	<code>var1 <= var2 var2 <= 5</code>



Tipos de operadores

Operadores aritméticos

Esses operadores podem ser velhos conhecidos se você já frequentou aulas de matemática elementar. São usados para fazer operações de soma, subtração, multiplicação, divisão, módulo e exponenciação.

Vamos explorar o uso desses operadores por meio de exemplos.



Tipos de operadores

Operadores aritméticos

somando dois operandos:

2 + 4; // o resultado será 6

Agora, a subtração:

4 - 2; // o resultado será 2

A multiplicação:

*2 * 4; // o valor da operação será 8*

E a divisão:

2 / 4 // como 2 dividido por 4 é 0.5, o resultado será esse valor.



Tipos de operadores

Operadores aritméticos

Para a exponenciação, a sintaxe é:

*4 ** 2 // retornará 16 que é 4 ao quadrado.*

operador de módulo (%) :

12 % 5; // o valor de resto será 2 que é o resto da divisão de 12 por 5.

Incremento (++) : Adiciona um ao seu operando. *3++ // resulta em 4*

Decremento (--) : Subtrai um de seu operando. *3-- // resulta em 2*

e Negação (-) : Retorna a negação de seu operando. *3 usando o operador fica -3*



Operadores aritméticos

Operador	Descrição	Exemplo
Módulo (%)	Operador binário. Retorna o inteiro restante da divisão dos dois operandos.	12 % 5 retorna 2.
Incremento (++)	Operador unário. Adiciona um ao seu operando. Se usado como operador prefixado (++x), retorna o valor de seu operando após a adição. Se usado como operador pósfixado (x++), retorna o valor de seu operando antes da adição.	Se x é 3, então ++x define x como 4 e retorna 4, enquanto x++ retorna 3 e, somente então, define x como 4.
Decremento (--)	Operador unário. Subtrai um de seu operando. O valor de retorno é análogo àquele do operador de incremento.	Se x é 3, então --x define x como 2 e retorna 2, enquanto x-- retorna 3 e, somente então, define x como 2.
Negação (-)	Operador unário. Retorna a negação de seu operando.	Se x é 3, então -x retorna -3.
Adição (+)	Operador unário. Tenta converter o operando em um número, sempre que possível.	+"3" retorna 3. +true retorna 1.
Operador de exponenciação (**)	Calcula a base elevada á potência do expoente, que é, base ^{expoente}	2 ** 3 retorna 8. 10 ** -1 retorna 0.1



Tipos de operadores

Operadores bit a bit

Os operadores bit a bit são utilizados para manipular valores para comparações e cálculos, comparando cada bit do primeiro operando com o correspondente do segundo.

Eles não são considerados operadores lógicos, já que os operandos são tratados como uma sequência de 32 bits, representados por zeros e uns, em vez de números decimais, hexadecimais ou octais.



Operadores bit a bit

Operador	Expressão	Descrição
AND	<code>a & b</code>	Retorna um 1 para cada posição em que os bits da posição correspondente de ambos operandos sejam uns.
OR	<code>a b</code>	Retorna um 0 para cada posição em que os bits da posição correspondente de ambos os operandos sejam zeros.
XOR	<code>a ^ b</code>	Retorna um 0 para cada posição em que os bits da posição correspondente são os mesmos. [Retorna um 1 para cada posição em que os bits da posição correspondente sejam diferentes.]
NOT	<code>~ a</code>	Inverte os bits do operando.
Deslocamento à esquerda	<code>a << b</code>	Desloca a em representação binária b bits à esquerda, preenchendo com zeros à direita.
Deslocamento à direita com propagação de sinal	<code>a >> b</code>	Desloca a em representação binária b bits à direita, descartando bits excedentes.
Deslocamento à direita com preenchimento zero	<code>a >>> b</code>	Desloca a em representação binária b bits à direita, descartando bits excedentes e preenchendo com zeros à esquerda.



Tipos de operadores

Operadores lógicos

São usados para realizar operações lógicas. Elas podem ser do tipo AND, OR e NOT. Os operandos devem ser lógicos, verdadeiro ou falso. Também podem operar sobre expressões lógicas, ou seja, que retornem valores verdadeiro ou falso.



Tipos de operadores

Operadores lógicos

AND (&&)

O operador AND (&&) recebe dois operandos e retorna verdadeiro se, e somente se ambos os operandos sejam verdadeiros. Retorna falso, caso contrário.

Por exemplo, a expressão:

true && true // retorna true.

Já a expressão:

true && false; // retorna false.



Tipos de operadores

Operadores lógicos

OR (||)

Para o operador OR (||) retornar verdadeiro, basta que um dos operandos seja verdadeiro. Ele também retorna verdadeiro caso os dois operandos sejam verdadeiros. Retorna falso, se os dois forem falsos.

A expressão: *true || true; // retorna true ;*

bem como a expressão: *true || false; // retorna false.*

Já a expressão: *false || false; // retorna false.*



Tipos de operadores

Operadores lógicos

NOT (!)

O operador de negação (!) é um operando unário, isto é, opera sobre apenas um operando. Ele nega, inverte o valor lógico do operando. Veja os exemplos:

var x = 2 > 1; // o valor de x é true

!x; // o valor retornado por essa expressão é false



Operadores lógicos

Operador	Utilização	Descrição
AND lógico (&&)	<code>expr1 && expr2</code>	(E lógico) - Retorna <code>expr1</code> caso possa ser convertido para falso; senão, retorna <code>expr2</code> . Assim, quando utilizado com valores booleanos, <code>&&</code> retorna verdadeiro caso ambos operandos sejam verdadeiros; caso contrário, retorna falso.
OU lógico ()	<code>expr1 expr2</code>	(OU lógico) - Retorna <code>expr1</code> caso possa ser convertido para verdadeiro; senão, retorna <code>expr2</code> . Assim, quando utilizado com valores booleanos, <code> </code> retorna verdadeiro caso ambos os operandos sejam verdadeiro; se ambos forem falsos, retorna falso.
NOT lógico (!)	<code>!expr</code>	(Negação lógica) Retorna falso caso o único operando possa ser convertido para verdadeiro; senão, retorna verdadeiro.



Tipos de operadores

Operadores de string

Concatenação (+)

O operador de concatenação é usado para unir strings em JavaScript. Este operador irá avaliar os valores dos operandos, que podem ser mais de dois neste caso, e retornar uma string que é a junção desses valores. Vale notar que os operandos não precisam necessariamente ser todos do tipo string.

O operador de concatenação (+) concatena dois valores string, retornando outra string que é a união dos dois operandos.

"minha " + "string"; // exibe a string "minha string".



Tipos de operadores

Operador condicional (ternário)

O operador condicional (ternário)

é o único operador JavaScript que utiliza três operandos. O operador pode ter um de dois valores baseados em uma condição. A sintaxe é: *condicao ? valor1 : valor2*.

Se *condicao* for verdadeira, o operador terá o valor de *valor1*. Caso contrário, terá o valor de *valor2*. podemos utilizar o operador condicional em qualquer lugar onde utilizaria um operador padrão.

```
var status = (idade >= 18) ? "adulto" : "menor de idade";
```



Tipos de operadores

Operador vírgula

A vírgula

É usada para denotar uma lista de valores. Ela pode ser usada para declarar várias variáveis de uma vez:

```
var x, y, z;
```

Para denotar elementos de uma lista:

```
var lista = [0, 1, 2, 3];
```



Tipos de operadores

Operador unário

Operadores unários

são aqueles que realizam uma operação sobre apenas um operador. Um exemplo de operador unário é o operador de negação (!) que irá inverter o valor lógico do único operador que precede.



Tipos de operadores

Operador typeof

O operador typeof

Retorna uma string indicando o tipo do operando sem avaliação. operando é uma string, variável, palavra-chave ou objeto cujo tipo deve ser retornado. Os parênteses são opcionais, é utilizado em qualquer uma das seguintes formas:

typeof operando
typeof (operando)



Tipos de operadores

Operador typeof

O operador typeof retornaria o seguinte resultado para aquelas variáveis:

```
typeof meuLazer(); // retorna "function"
```

```
typeof "forma"; // retorna "string"
```

```
typeof 36; // retorna "number"
```

```
typeof 12.6; // retorna "number"
```

```
typeof {hoje}; // retorna "object"
```

```
typeof naoExiste; // retorna "undefined"
```

```
typeof true; // retorna "boolean"
```

```
typeof null; // retorna "object"
```



Tipos de operadores

Operadores relacionais

in

É usado para definir se um objeto tem uma determinada propriedade. Ele retorna os valores true ou false. Um uso bastante comum do operador in é para verificar se um elemento pertence a um array:

1 in [1,2,3,4]; // retorna true pois 1 pertence ao array

1 in []; // retorna false pois o valor 1 não está no array (que é vazio)



Tipos de operadores

Operadores relacionais

Instanceof

outro operador relacional, retorna verdadeiro se um objeto é uma instância de outro objeto. Caso contrário, retorna falso.

nomeObjeto instanceof tipoObjeto

Utilize o instanceof quando você precisar confirmar o tipo de um objeto em tempo de execução. Por exemplo, ao capturar exceções você pode desviar para um código de manipulação de exceção diferente dependendo do tipo de exceção lançada.



Tipos de operadores

Operadores relacionais

Por exemplo, o código a seguir utiliza o `instanceof` para determinar se `dia` é um objeto `Date`. Como `dia` é um objeto `Date`, as declarações do `if` são executadas.

```
var dia = new Date(1995, 12, 17);  
if (dia instanceof Date) {  
    // declarações a serem executadas  
}
```